



Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😊 +

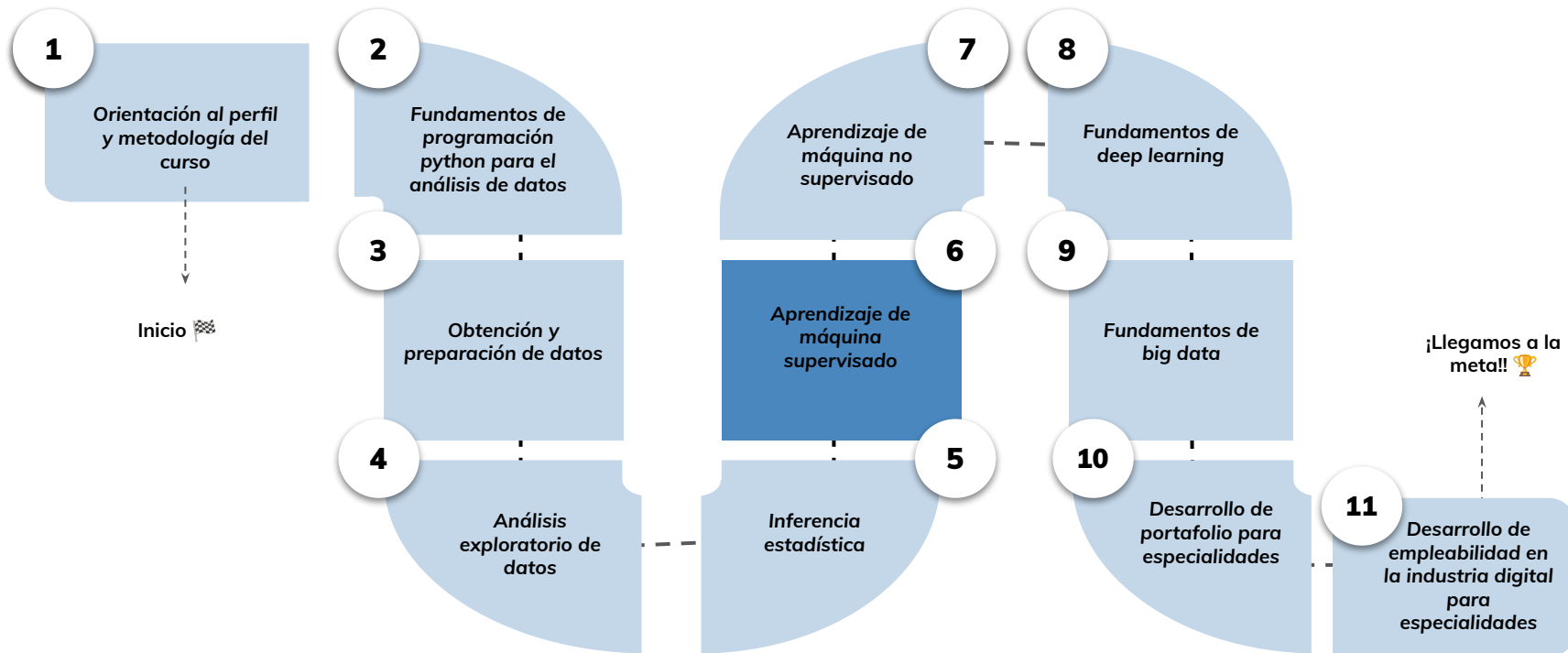


Optimización de un modelo- › Parte II

Aprendizaje Esperado 7: Aplicar técnicas de optimización de un modelo de aprendizaje para la mejora de su desempeño acorde a las buenas prácticas de la industria.

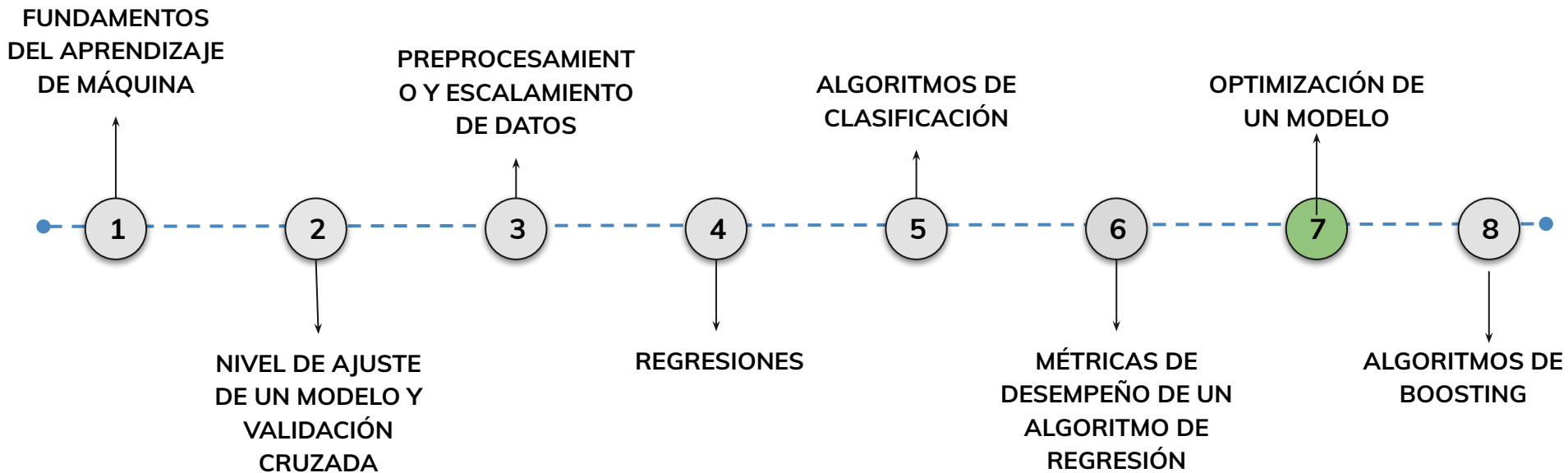
Hoja de ruta

¿Cuáles skills conforman el programa? **Fundamentos de Ciencia de Datos**



Roadmap de lecciones

¿Cuáles **lecciones** estaremos estudiando en este módulo?



Learning Path

¿Cuáles temas trabajaremos hoy?

7.

Optimización inteligente de hiperparámetros

exploraremos cómo usar búsqueda bayesiana con Optuna para encontrar automáticamente la mejor configuración del modelo.

¿Qué es Optuna y cómo funciona?

Espacio de búsqueda y función objetivo

Comparación con Grid y Random Search

Ensamblado de modelos

Bagging, Boosting, Stacking

Cuándo usar cada técnica

Objetivos de aprendizaje

¿Qué aprenderás?

- Explorar técnicas avanzadas para mejorar el rendimiento de modelos.
- Aplicar Optuna para optimización bayesiana de hiperparámetros.
- Implementar métodos de ensamblado como bagging, boosting y stacking.
- Evaluar modelos con validación robusta: nested CV, bootstrapping, múltiples semillas.
- Introducir conceptos de AutoML y aprendizaje por refuerzo para optimización.

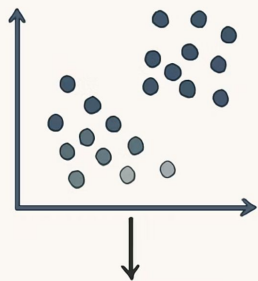
Repaso clase anterior

¿Quedó alguna duda?

En la clase anterior trabajamos :

- Cómo estructurar pipelines completos con Scikit-learn.
- La optimización de hiperparámetros con GridSearchCV.
- Ventajas del enfoque modular para evitar errores comunes.
- Métricas relevantes para problemas de clasificación y regresión.

Optimización de un modelo



Data
Preprocessing

Model
Training

Prediction

```
import sklearn
```

```
X, y = load_iris(return_X_y=True)
```

```
clf = RandomForestClassifier()
```

```
clf.fit(X, y)
```

```
y_pred = clf.predict(X)
```

Implementación con la librería Scikit-learn

La librería **Scikit-learn** es una de las herramientas más utilizadas en Python para la creación y optimización de modelos de aprendizaje automático. Su diseño simple y coherente permite construir flujos de trabajo robustos que abarcan desde el preprocesamiento de datos hasta la validación y evaluación de modelos. Gracias a su compatibilidad con otras bibliotecas del ecosistema Python, como NumPy, pandas y Matplotlib, es una opción versátil y poderosa tanto para tareas experimentales como productivas.

Ventajas del enfoque modular de Scikit-learn

Una de las principales ventajas de Scikit-learn es su enfoque modular. Todos los estimadores comparten una API común, lo que permite construir **pipelines** en los que se integran transformadores y modelos de forma encadenada. Esto facilita la organización del código, mejora la reproducibilidad y evita la fuga de datos durante las fases de entrenamiento y validación cruzada.

Capacidades de Scikit-learn



Técnicas integradas

En Scikit-learn se pueden aplicar fácilmente técnicas de regularización, ajuste de hiperparámetros, selección de características, validación cruzada y balanceo de clases.



Algoritmos eficientes

Incluye implementaciones eficientes de algoritmos como regresión logística, árboles de decisión, SVM, bosques aleatorios, k-vecinos, entre otros. Todo esto con documentación clara y ejemplos accesibles.



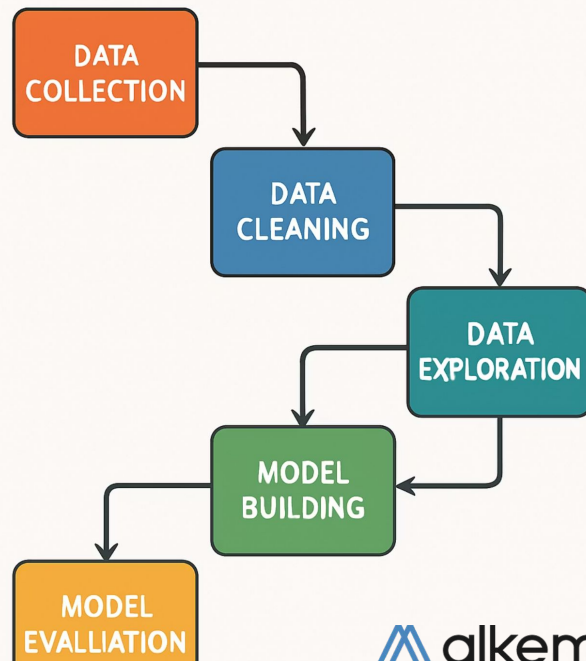
Optimización integrada

La integración de GridSearchCV y RandomizedSearchCV permite optimizar hiperparámetros directamente sobre un pipeline, combinando preprocesamiento y modelo en un solo objeto.

Beneficios de los pipelines en Scikit-learn

Esto no solo mejora la legibilidad, sino que reduce errores comunes al ajustar manualmente etapas por separado. También es posible evaluar el rendimiento con múltiples métricas y seleccionar el mejor modelo según la métrica más relevante para el negocio.

DATA SCIENCE PIPELINE



Ejemplo de pipeline completo

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import GridSearchCV
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split

# Generar datos desbalanceados
X, y = make_classification(n_samples=1000, n_features=20, weights=[0.8, 0.2], random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)

# Crear pipeline con escalado y modelo
pipeline = Pipeline([ ('scaler', StandardScaler()), ('classifier', LogisticRegression(random_state=42)) ])

# Definir grilla de hiperparámetros
param_grid = { 'classifier__C': [0.01, 0.1, 1, 10], 'classifier__penalty': ['l1', 'l2'], 'classifier__class_weight': [None, 'balanced'] }

# Búsqueda en grilla con validación cruzada
grid_search = GridSearchCV(pipeline, param_grid, cv=5, scoring='f1')
grid_search.fit(X_train, y_train)

# Mejores hiperparámetros y rendimiento
print(f"Mejores parámetros: {grid_search.best_params_}")
print(f"Mejor puntuación F1: {grid_search.best_score_: .4f}")

# Evaluar en conjunto de prueba
y_pred = grid_search.predict(X_test)

from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred))
```

Beneficios del enfoque profesional

Este ejemplo muestra cómo Scikit-learn permite construir un flujo completo y profesional en pocas líneas de código. La correcta implementación de estas herramientas reduce los errores, mejora la productividad del equipo de datos y asegura que los modelos sean confiables para su uso en producción.



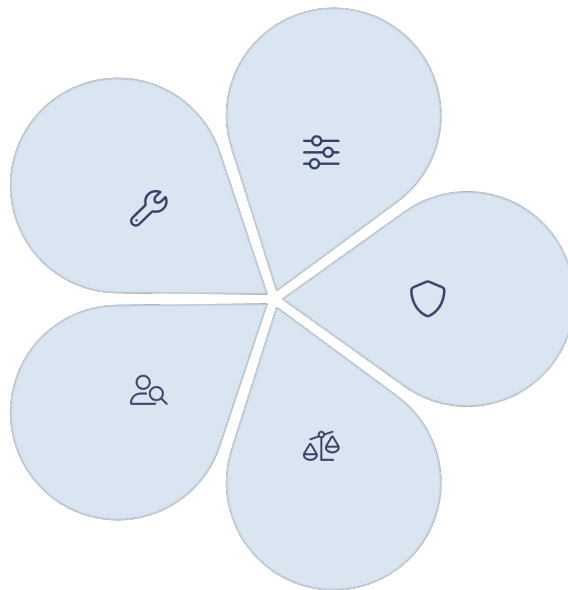
Resumen de técnicas de optimización

Feature Engineering

Selección, transformación y creación de variables predictoras.

Búsqueda en grilla

Exploración sistemática del espacio de hiperparámetros.



Ajuste de hiperparámetros

Configuración óptima de valores que controlan el entrenamiento.

Regularización

Control de la complejidad para evitar el sobreajuste.

Balanceo de datos

Técnicas para manejar clases desbalanceadas.

Importancia de la diferenciación entre parámetros e hiperparámetros

A lo largo de este módulo hemos explorado los principales mecanismos para optimizar modelos de aprendizaje automático, desde conceptos teóricos fundamentales hasta su implementación práctica con la biblioteca Scikit-learn. Cada técnica —Feature Engineering, ajuste de hiperparámetros, regularización, balanceo de datos y búsqueda en grilla— cumple un rol esencial para mejorar el desempeño predictivo y garantizar modelos más robustos, escalables y confiables.

Comprender la diferencia entre parámetros e hiperparámetros permite tomar decisiones más informadas al momento de configurar modelos. Asimismo, aplicar técnicas de regularización ayuda a encontrar un equilibrio entre complejidad y generalización, mientras que el uso de métodos de búsqueda sistemática, como Grid Search, asegura un entrenamiento más riguroso y objetivo.

PARAMETERS	HYPERPARAMETERS
Learned automatically	Set manually
Values estimated from data	Values specified before training
Model weights and biases	Learning rate, number of epochs, etc
Depend on the algorithm	Depend on the model and training process

Balanceo de clases en escenarios reales

El balanceo de clases es una práctica indispensable en escenarios reales donde los datos suelen estar desbalanceados. Este aspecto, muchas veces subestimado, puede ser decisivo en la efectividad del modelo, especialmente en contextos críticos como salud, seguridad o finanzas. La capacidad de implementar estas soluciones con herramientas accesibles como Scikit-learn potencia el trabajo de los profesionales de datos.

Beneficios de los pipelines en entornos productivos

Además, la integración de todo el flujo mediante pipelines facilita la reproducibilidad y trazabilidad, aspectos clave en entornos productivos. La automatización del preprocesamiento, validación y ajuste de parámetros se traduce en eficiencia operativa y calidad del producto final. En un mundo donde los modelos de aprendizaje automático deben evolucionar constantemente, contar con flujos bien estructurados es una ventaja competitiva.

Profundizando en Feature Engineering

El Feature Engineering es quizás uno de los aspectos más creativos y a la vez técnicos del aprendizaje automático. Requiere tanto conocimiento del dominio como intuición estadística para identificar qué transformaciones pueden revelar patrones ocultos en los datos. Algunas técnicas avanzadas incluyen:

- Descomposición de variables temporales (extraer día de la semana, mes, trimestre)
- Transformaciones no lineales (logaritmos, raíces cuadradas)
- Codificación de variables categóricas de alta cardinalidad
- Creación de características basadas en agrupaciones (promedios por grupo)



Técnicas avanzadas de regularización

Dropout

Aunque se asocia principalmente con redes neuronales, el concepto de "apagar" aleatoriamente algunas unidades durante el entrenamiento puede adaptarse a otros modelos como forma de regularización.

Early Stopping

Detener el entrenamiento cuando el rendimiento en un conjunto de validación comienza a deteriorarse, evitando así el sobreajuste por exceso de iteraciones.

Regularización por restricciones

Limitar directamente la complejidad del modelo, por ejemplo, restringiendo la profundidad máxima de un árbol o el número de componentes en un modelo PCA.

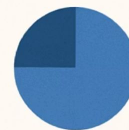
Métricas de evaluación según el contexto

La elección de la métrica adecuada para evaluar un modelo es tan importante como el modelo mismo. Dependiendo del problema, algunas métricas son más relevantes que otras:

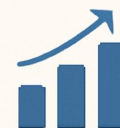
- Para problemas de clasificación desbalanceados: F1-score, precisión-recall AUC
- Para sistemas de recomendación: NDCG, MAP
- Para modelos de regresión: RMSE, MAE, R^2
- Para detección de anomalías: precisión, recall, especificidad

Es fundamental alinear la métrica de optimización con el objetivo de negocio real.

EVALUATION METRICS



ACCURACY



PRECISION



F1-SCORE



RECALL



ROC AUC



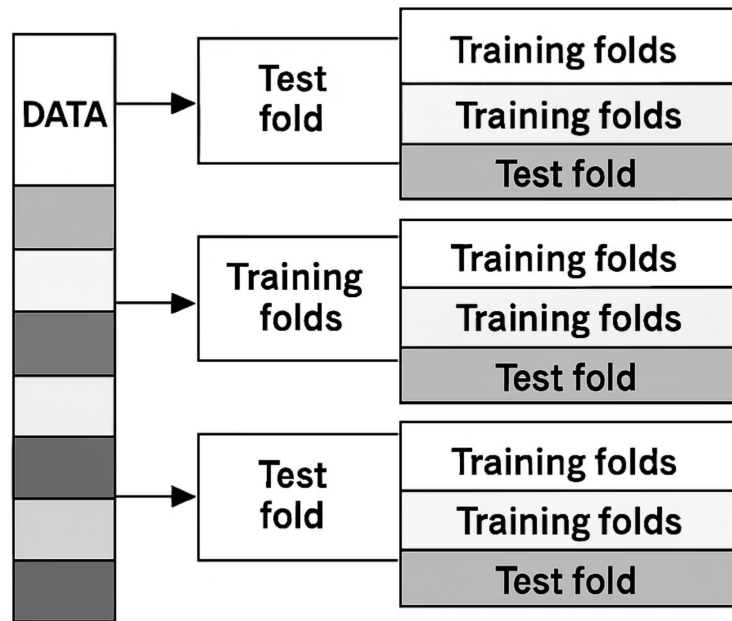
LOG LOSS

Validación cruzada estratificada

La validación cruzada estratificada es una variante de la validación cruzada tradicional que mantiene la proporción de clases en cada fold. Esto es especialmente importante en conjuntos desbalanceados, donde una división aleatoria podría resultar en folds sin ejemplos de la clase minoritaria.

```
from sklearn.model_selection import StratifiedKFoldfrom sklearn.metrics import
accuracy_scoreimport numpy as np# Configurar validación cruzada estratificadaskf =
StratifiedKFold(n_splits=5, shuffle=True, random_state=42)# Iterar sobre los foldsscores =
[]for train_idx, test_idx in skf.split(X, y): X_train_fold, X_test_fold = X[train_idx],
X[test_idx] y_train_fold, y_test_fold = y[train_idx], y[test_idx] # Entrenar y evaluar
modelo model.fit(X_train_fold, y_train_fold) y_pred = model.predict(X_test_fold)
scores.append(accuracy_score(y_test_fold, y_pred))print(f"Puntuación media:
{np.mean(scores):.4f}")print(f"Desviación estándar: {np.std(scores):.4f}")
```

STRATIFIED CROSS-VALIDATION



Técnicas de ensamblado

Los métodos de ensamblado combinan múltiples modelos para obtener predicciones más robustas y precisas. Las principales técnicas son:

1

Bagging

Entrena varios modelos en diferentes submuestras aleatorias del conjunto de datos (con reemplazo) y promedia sus predicciones. Random Forest es un ejemplo de bagging aplicado a árboles de decisión.

2

Boosting

Entrena modelos secuencialmente, donde cada nuevo modelo se enfoca en los errores de los anteriores. Ejemplos incluyen AdaBoost, Gradient Boosting y XGBoost.

3

Stacking

Combina las predicciones de varios modelos base usando un "meta-modelo" que aprende cómo integrar estas predicciones de manera óptima.

Optimización de hiperparámetros con Optuna

Optuna es una biblioteca de optimización de hiperparámetros que utiliza técnicas bayesianas y de poda para encontrar configuraciones óptimas de manera eficiente. A diferencia de GridSearchCV, Optuna puede:

- Definir espacios de búsqueda complejos y condicionales
- Eliminar automáticamente pruebas poco prometedoras (poda)
- Visualizar la importancia de los hiperparámetros
- Paralelizar la búsqueda de forma eficiente

```
import optuna
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

def objective(trial):
    # Definir espacio de búsqueda
    n_estimators = trial.suggest_int('n_estimators', 10, 200)
    max_depth = trial.suggest_int('max_depth', 2, 32, log=True)
    min_samples_split = trial.suggest_float('min_samples_split', 0.01, 0.5)

    # Crear y entrenar modelo
    clf = RandomForestClassifier(
        n_estimators=n_estimators,
        max_depth=max_depth,
        min_samples_split=min_samples_split,
        random_state=42
    )
    clf.fit(X_train, y_train)

    # Evaluar
    y_pred = clf.predict(X_valid)
    return accuracy_score(y_valid, y_pred)

# Ejecutar optimización
study = optuna.create_study(direction='maximize')
study.optimize(objective, n_trials=100)

print(f"Mejores hiperparámetros: {study.best_params}")
print(f"Mejor puntuación: {study.best_value:.4f}")
```


Detección y tratamiento de valores atípicos

Los valores atípicos (outliers) pueden afectar significativamente el rendimiento de muchos algoritmos, especialmente aquellos basados en distancias o medias. Algunas estrategias para manejarlos incluyen:

- Detección mediante métodos estadísticos (Z-score, IQR)
- Algoritmos específicos como Isolation Forest o One-Class SVM
- Transformaciones robustas como winsorización o escalado robusto
- Eliminación selectiva cuando es apropiado

OUTLIER DETECTION METHODS

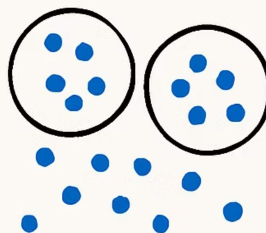
Z-Score



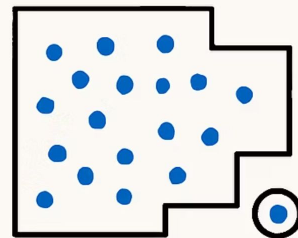
Modified Z-Score



DBSCAN



Isolation Forest



Interpretabilidad de modelos

La interpretabilidad es cada vez más importante, especialmente en sectores regulados. Algunas técnicas para entender el comportamiento de los modelos incluyen:

SHAP Values

Basados en teoría de juegos, calculan la contribución de cada característica a la predicción final.

LIME

Crea aproximaciones locales interpretables alrededor de predicciones específicas.

Gráficos de dependencia parcial

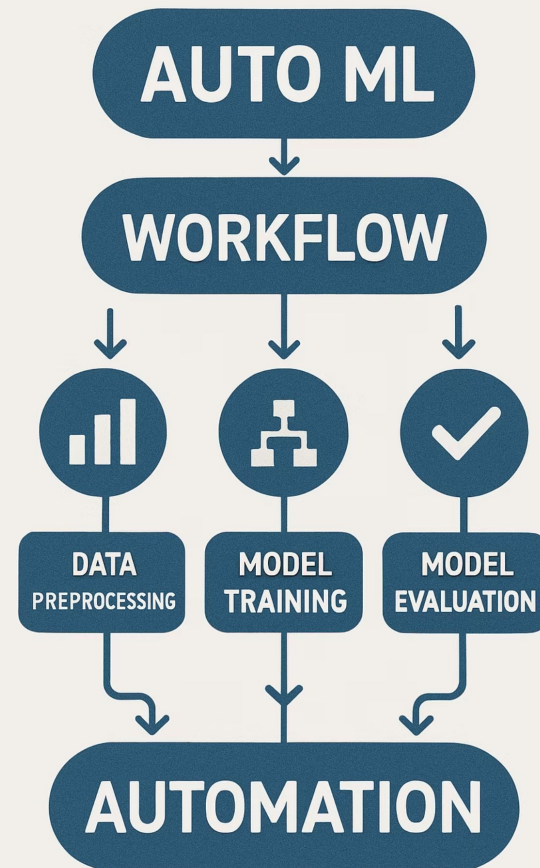
Muestran cómo cambia la predicción al variar una característica mientras se mantienen las demás constantes.

Automatización del aprendizaje automático (AutoML)

Las herramientas de AutoML automatizan gran parte del proceso de creación de modelos, incluyendo:

- Selección automática de características
- Prueba de múltiples algoritmos
- Optimización de hiperparámetros
- Ensamblado de modelos

Bibliotecas como Auto-Sklearn, TPOT y H2O AutoML permiten crear modelos competitivos con mínima intervención humana, aunque el conocimiento experto sigue siendo valioso para problemas complejos o específicos del dominio.



Validación temporal para series temporales

Los datos temporales requieren técnicas especiales de validación para evitar fugas de información del futuro al pasado:

- 1 Validación forward**
Entrena con datos hasta un punto t y valida en $t+1$, luego mueve la ventana.
- 2 Validación expanding window**
Incrementa gradualmente el conjunto de entrenamiento manteniendo fijo el inicio.
- 3 Validación rolling origin**
Mantiene constante el tamaño de la ventana de entrenamiento, moviéndola completamente.

Scikit-learn proporciona `TimeSeriesSplit` para implementar estas estrategias de forma sencilla.

Optimización de modelos para producción

Preparar un modelo para producción implica consideraciones adicionales:

- Serialización eficiente (pickle, joblib, ONNX)
- Reducción de tamaño y complejidad (poda de árboles, cuantización)
- Monitoreo de drift y degradación del rendimiento
- Estrategias de reentrenamiento (periódico, basado en rendimiento)
- Integración con sistemas de CI/CD para MLOps

Técnicas de validación robusta

1

Nested Cross-Validation

Utiliza un bucle externo de validación cruzada para evaluar el rendimiento general, y un bucle interno para optimizar hiperparámetros. Esto proporciona una estimación más realista del rendimiento en datos no vistos.

2

Bootstrapping

Genera múltiples muestras con reemplazo del conjunto de datos original y evalúa el modelo en cada una, obteniendo intervalos de confianza para las métricas de rendimiento.

3

Validación con múltiples semillas

Repite el proceso de entrenamiento y evaluación con diferentes semillas aleatorias para asegurar que los resultados son estables y no dependen de una inicialización particular.

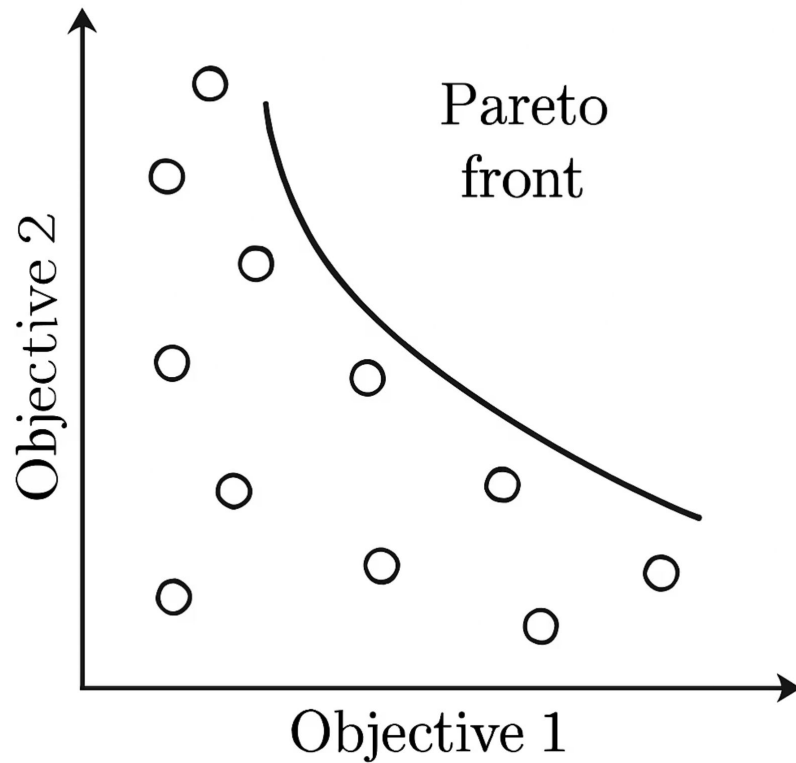
Selección de características avanzada

Más allá de los métodos básicos, existen técnicas sofisticadas para seleccionar las características más relevantes:

- Recursive Feature Elimination (RFE): elimina iterativamente las características menos importantes
- Selección basada en modelos (feature_importances_ en árboles)
- Métodos de filtro basados en correlación o información mutua
- Métodos wrapper que evalúan subconjuntos de características
- Técnicas embebidas como Lasso que realizan selección durante el entrenamiento

Optimización multi-objetivo

En muchos casos prácticos, se busca optimizar múltiples métricas simultáneamente, como precisión y tiempo de inferencia, o sensibilidad y especificidad. Las técnicas de optimización multi-objetivo permiten encontrar soluciones de compromiso (frente de Pareto) entre objetivos contrapuestos.



Calibración de probabilidades

Muchos modelos de clasificación producen probabilidades que no están bien calibradas (no reflejan la verdadera probabilidad de pertenencia a una clase). La calibración de probabilidades ajusta estas estimaciones para que sean más precisas:

- Calibración de Platt: ajusta las probabilidades mediante regresión logística
- Calibración isotónica: ajuste no paramétrico más flexible
- Calibración por bins: agrupa predicciones similares y ajusta por grupo

Un modelo bien calibrado es crucial para aplicaciones que requieren estimaciones de riesgo precisas, como medicina o finanzas.

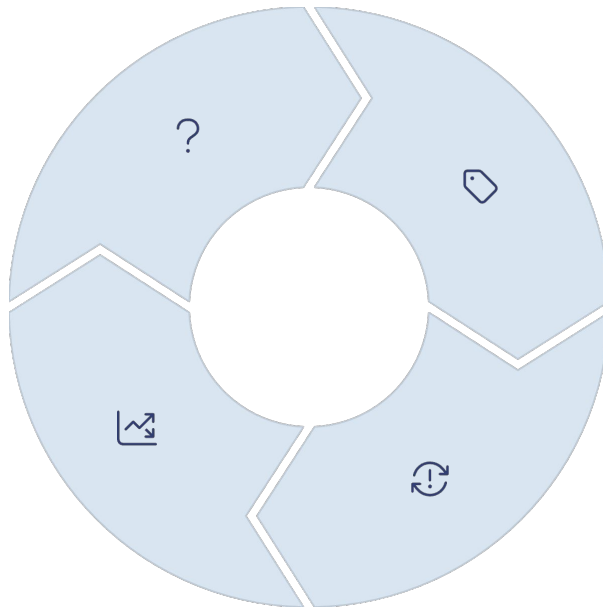
Aprendizaje activo

Selección de muestras

Identificar los ejemplos más informativos para etiquetar

Evaluación

Medir el rendimiento del modelo actualizado



Etiquetado

Obtener etiquetas para los ejemplos seleccionados

Reentrenamiento

Actualizar el modelo con los nuevos datos etiquetados

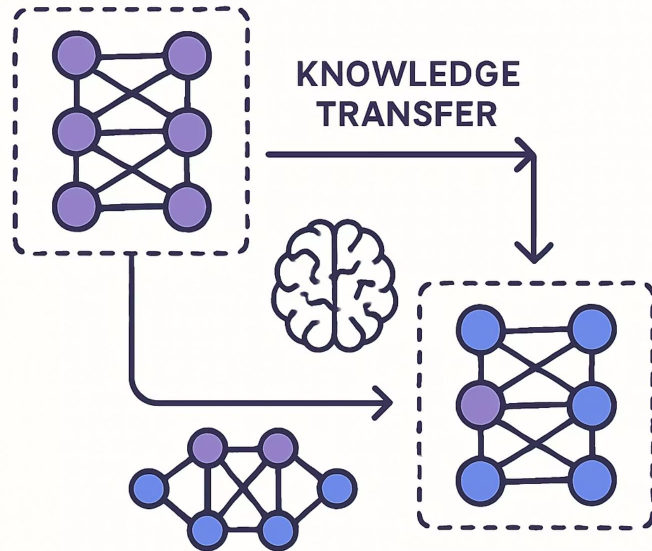
El aprendizaje activo es especialmente útil cuando el etiquetado de datos es costoso o lento, ya que permite maximizar el rendimiento con un mínimo de datos etiquetados.

Transferencia de aprendizaje

La transferencia de aprendizaje aprovecha el conocimiento adquirido en una tarea para mejorar el rendimiento en otra relacionada. Aunque es más conocida en deep learning, también se aplica en modelos tradicionales:

- Reutilización de características extraídas de modelos pre-entrenados
- Inicialización de parámetros basada en modelos similares
- Adaptación de dominio para generalizar entre diferentes distribuciones
- Few-shot learning para aprender con pocos ejemplos

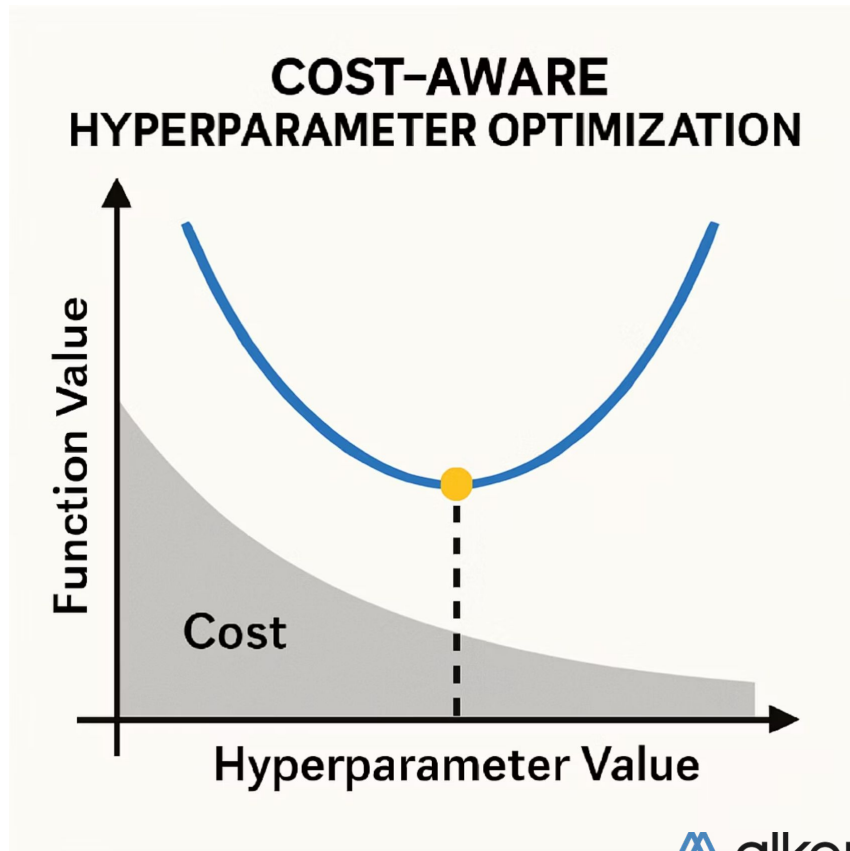
TRANSFER LEARNING



Optimización de hiperparámetros sensible al costo

No todos los hiperparámetros tienen el mismo impacto en el rendimiento o el costo computacional. Las técnicas de optimización sensibles al costo consideran tanto el rendimiento como los recursos necesarios:

- Priorización de hiperparámetros más influyentes
- Presupuestos de tiempo o recursos para la optimización
- Consideración de compromisos entre precisión y eficiencia



Detección y corrección de sesgos

Los modelos pueden perpetuar o amplificar sesgos presentes en los datos de entrenamiento. Las técnicas para abordar este problema incluyen:

Preprocesamiento

Reequilibrar o transformar los datos para reducir sesgos antes del entrenamiento

Durante el entrenamiento

Incorporar restricciones de equidad en la función objetivo

Postprocesamiento

Ajustar las predicciones para garantizar resultados equitativos entre grupos

La detección y mitigación de sesgos es especialmente importante en aplicaciones con impacto social, como contratación, préstamos o justicia criminal.

Aprendizaje por refuerzo para optimización de hiperparámetros

El aprendizaje por refuerzo puede utilizarse para optimizar hiperparámetros de forma más eficiente que los métodos tradicionales:

- Modelado del proceso de optimización como un problema de decisión secuencial
- Aprendizaje de estrategias eficientes para explorar el espacio de hiperparámetros
- Adaptación dinámica basada en resultados previos
- Transferencia de conocimiento entre tareas de optimización similares

Bibliotecas como Ray Tune implementan variantes de estos enfoques para acelerar la búsqueda de configuraciones óptimas.



Conclusión: El futuro de la optimización de modelos

La optimización de modelos de aprendizaje automático es un campo en constante evolución, con tendencias emergentes como:

- Optimización neural (utilizando redes neuronales para predecir buenos hiperparámetros)
- Meta-aprendizaje (aprender a optimizar a partir de tareas anteriores)
- Optimización diferenciable de arquitecturas
- Sistemas completamente automatizados de extremo a extremo

Sin embargo, el conocimiento humano sigue siendo invaluable para guiar estos procesos, especialmente en la definición de problemas, la ingeniería de características creativa y la interpretación de resultados en el contexto del dominio específico.

Live Coding

¿En qué consistirá la Demo?

Vamos a entrenar un modelo de clasificación usando Random Forest, optimizando hiperparámetros con Optuna, y luego analizaremos la importancia de cada uno y la performance final.

1. Cargar y preprocesar el dataset (make_classification o un CSV propio).
2. Definir función objetivo para Optuna (accuracy sobre validación).
3. Explorar el espacio: n_estimators, max_depth, min_samples_split.
4. Visualizar los mejores parámetros y rendimiento.
5. Calibrar probabilidades con CalibratedClassifierCV.
6. Usar SHAP para interpretar las predicciones finales.



Momento:

Time-out!



5 -10 min.





Ejercicio

Optimizando como profesionales

Optimizando como profesionales

Contexto: 🙌

Estás trabajando en un equipo de ciencia de datos que entrena modelos de clasificación para detectar fraudes bancarios. Los datos están fuertemente desbalanceados, y el costo de falsos negativos es crítico. Quieren probar enfoques avanzados para optimizar modelos con múltiples objetivos.

Consigna: 📝

1. Cargar el dataset fraude.csv o uno generado con `make_classification`.
2. Implementar un pipeline con preprocesamiento (StandardScaler) y modelo (RandomForestClassifier).
3. Optimizar hiperparámetros usando Optuna, evaluando `f1_score`.
4. Agregar calibración de probabilidades (CalibratedClassifierCV).
5. Realizar interpretación con SHAP.
6. Aplicar una técnica de ensamblado (Boosting o Stacking) como alternativa.
7. Comentar cuál estrategia tuvo mejor rendimiento y justificar.

Paso a paso: ⚙️

1. Preprocesamiento y división de datos
2. Configuración de Optuna
3. Entrenamiento y evaluación
4. Calibración
5. Interpretabilidad
6. Ensamblado
7. Conclusiones y reflexión

Tiempo 🕒: 45 Minutos

¿Alguna consulta?





Resumen

¿Qué logramos en esta clase?



- ✓ **Aplicamos técnicas avanzadas de optimización de modelos.**
- ✓ **Utilizamos Optuna para búsquedas inteligentes.**
- ✓ **Comprendimos el valor de calibrar probabilidades.**
- ✓ **Aplicamos ensamblado de modelos para mejorar precisión.**
- ✓ **Introducimos AutoML y aprendizaje por refuerzo como próximos pasos.**



¡Ponte a prueba!

Momento de ejercitación

Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las **actividades disponibles en la plataforma**. Estas propuestas son clave para afianzar lo trabajado y **forman parte obligatoria del recorrido de aprendizaje**.

👉 **Análisis de caso** ————— 👉 **Selección Múltiple**

👉 **Comprensión lectora**

Si al resolverlas surge alguna duda, compartela o tráela al próximo encuentro sincrónico.

< **¡Muchas gracias!** >

